LAPORAN PRAKTIKUM
Praktikum 7 Exception Handling

Diajukan untuk memenuhi salah satu tugas praktikum Mata kuliah Pemrograman Berorientasi Objek



Disusun Oleh:
Radithya Mahesa Syabil (251511025)
Jurusan Teknik Komputer dan Informatika

Program Studi D-3 Teknik Informatika
Politeknik Negeri Bandung
2026

A. CountLetters (Exception Tidak Selalu Berarti Error)

1.1 Kode Program Awal

Program ini menghitung berapa kali tiap huruf muncul di sebuah kata. Hasil hitungannya disimpan di array counts, satu slot untuk satu huruf.

```
import java.util.Scanner;

public class CountLetters {
    public static void main(String[] args) {
        int[] counts = new int[26];
        Scanner scan = new Scanner(System.in);

        System.out.print("Enter a single word (letters only,
please): ");
        String word = scan.nextLine();

        word = word.toUpperCase();

        for (int i = 0; i < word.length(); i++)
            counts[word.charAt(i) - 'A']++;

        System.out.println();
        for (int i = 0; i < counts.length; i++)
            if (counts[i] != 0)
                System.out.println((char)(i + 'A') + ": " +
counts[i]);
    }
}
```

1.2 Uji Input Normal (T7-01)

Saya coba dulu pakai kata JAVA dan hasilnya sudah sesuai, tiap huruf terhitung dengan benar.



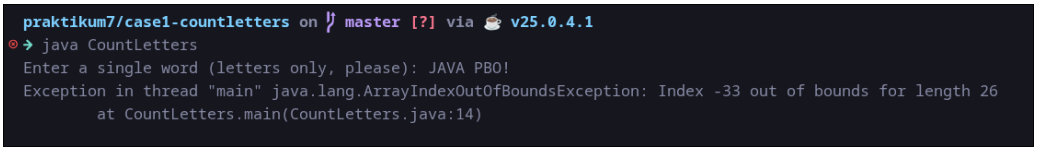
Gambar 1. Eksekusi normal CountLetters dengan input JAVA

- Checkpoint 1

Ekspresi word.charAt(i) - 'A' bekerja lewat kode ASCII. Karakter 'A' bernilai 65, jadi 'A' - 'A' hasilnya 0 dan 'Z' - 'A' hasilnya 25. Array counts dibuat berukuran 26 karena huruf alfabet dari A sampai Z memang ada 26.

1.3 Munculnya ArrayIndexOutOfBoundsException (T7-02)

Selanjutnya saya masukkan JAVA PBO!. Karakter ke-4 adalah spasi yang kode ASCII-nya 32, sehingga 32 - 65 menghasilkan -33. Angka ini jelas di luar rentang indeks array yaitu 0 sampai 25, jadi programnya langsung crash.



Gambar 2. ArrayIndexOutOfBoundsException saat input mengandung spasi

- Checkpoint 2

Alur program terputus begitu exception muncul di baris 14. Akibatnya loop kedua yang tugasnya mencetak frekuensi huruf tidak pernah sempat dijalankan.

1.4 Percobaan Try-Catch Kosong (Step 3)

Di percobaan ini blok try-catch saya taruh di dalam loop pertama dengan isi catch dikosongkan. Hasilnya program tidak lagi berhenti waktu ketketemu spasi atau tanda seru, dan huruf sesudahnya (P, B, O) tetap terhitung.

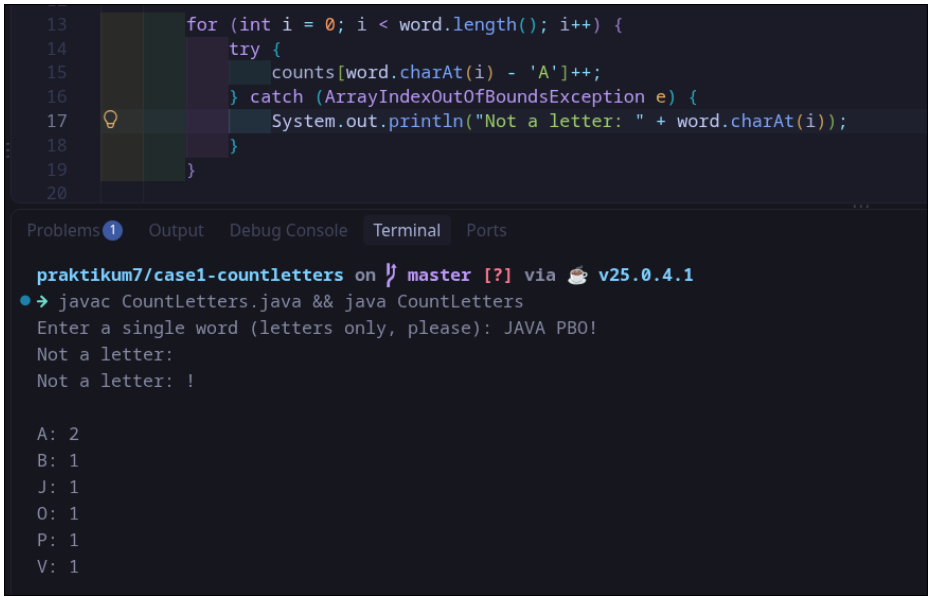


Gambar 3. Hasil percobaan try-catch kosong

1.5 Handler yang Lebih Informatif (T7-03)

Supaya lebih jelas, isi catch saya ganti agar menampilkan karakter non-huruf yang dilewati.

```
for (int i = 0; i < word.length(); i++) {
    try {
        counts[word.charAt(i) - 'A']++;
    } catch (ArrayIndexOutOfBoundsException e) {
        System.out.println("Not a letter: " + word.charAt(i));
    }
}
```



Gambar 4. Handler informatif pada CountLetters

- Checkpoint 3

Sebelum ada catch, program langsung berhenti di karakter pertama yang bukan huruf dan karakter sesudahnya tidak ikut dihitung. Setelah ada catch, exception ditangani di setiap iterasi sehingga program bisa pulih dan lanjut ke karakter berikutnya.

B. ParseInts (Penempatan Exception Handler)

2.1 Kode Program Awal

Program ini membaca satu baris teks lalu menjumlahkan semua token yang berupa bilangan bulat.

```
import java.util.Scanner;

public class ParseInts {
    public static void main(String[] args) {
        int val, sum = 0;
        Scanner scan = new Scanner(System.in);

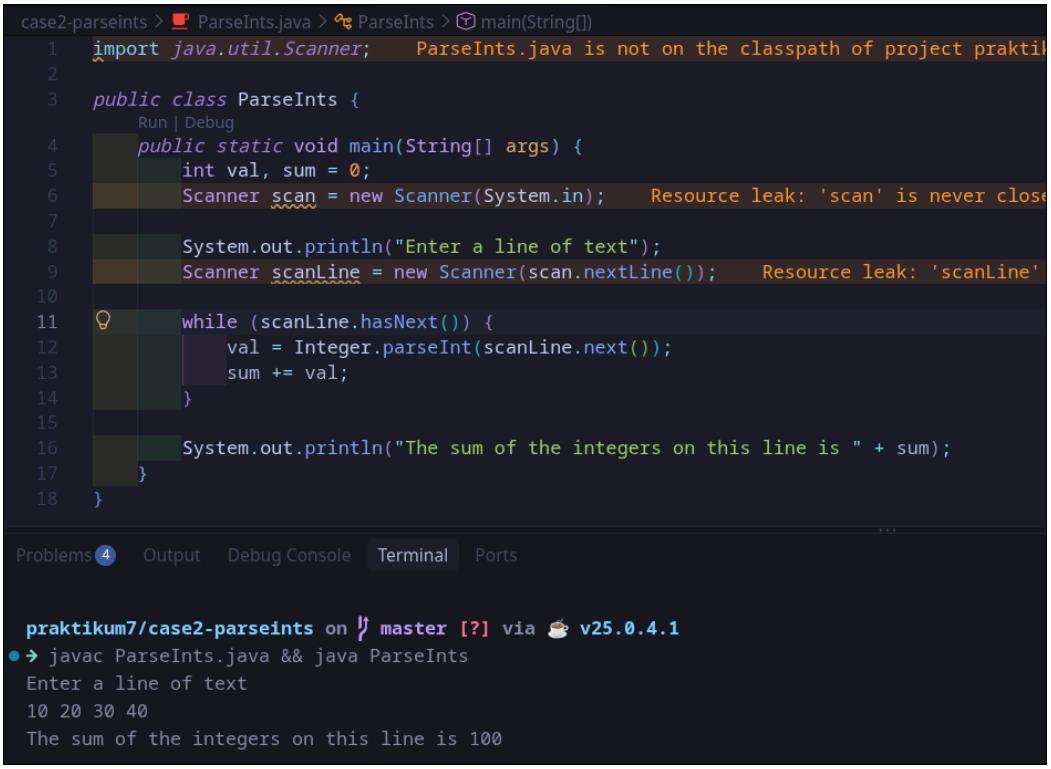
        System.out.println("Enter a line of text");
        Scanner scanLine = new Scanner(scan.nextLine());

        while (scanLine.hasNext()) {
            val = Integer.parseInt(scanLine.next());
            sum += val;
        }

        System.out.println("The sum of the integers on this line
is " + sum);
    }
}
```

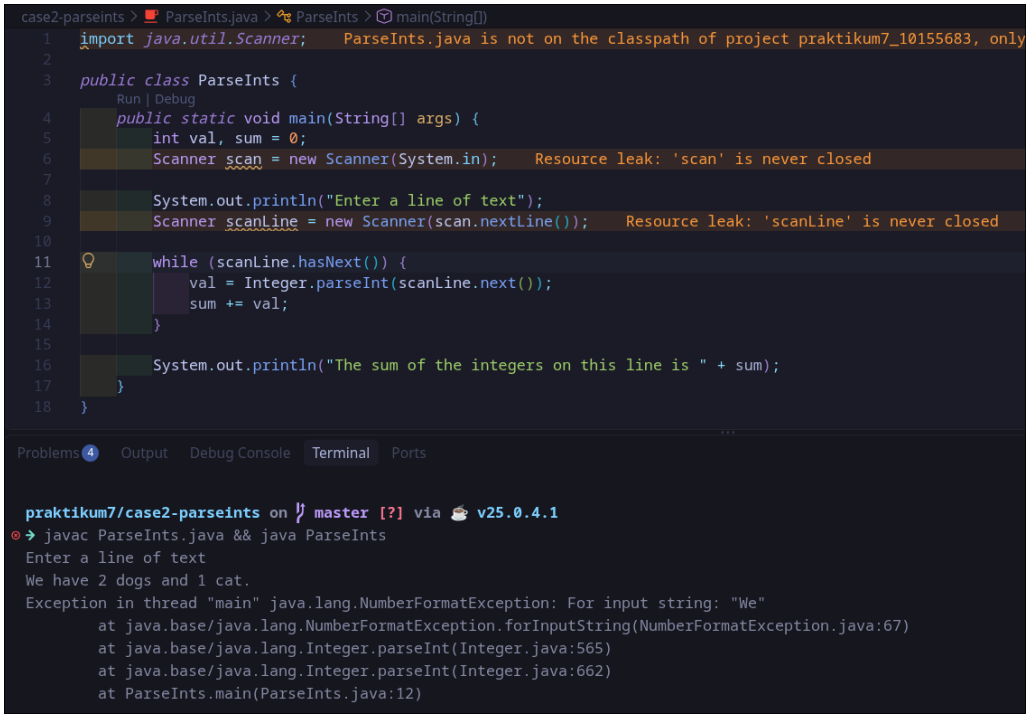
2.2 Uji Input Valid dan Input Teks

Dengan input 10 20 30 40, hasil penjumlahannya 100 dan semuanya berjalan normal.



Gambar 5. ParseInts dengan input angka valid

Kalau inputnya We have 2 dogs and 1 cat., program crash dengan NumberFormatException.



Gambar 6. NumberFormatException pada ParseInts

- Checkpoint 4

Method yang melempar exception adalah Integer.parseInt(). Pemicunya token teks pertama, yaitu "We", karena tidak bisa diubah jadi bilangan bulat.

2.3 Try-Catch di Luar While (T7-04)

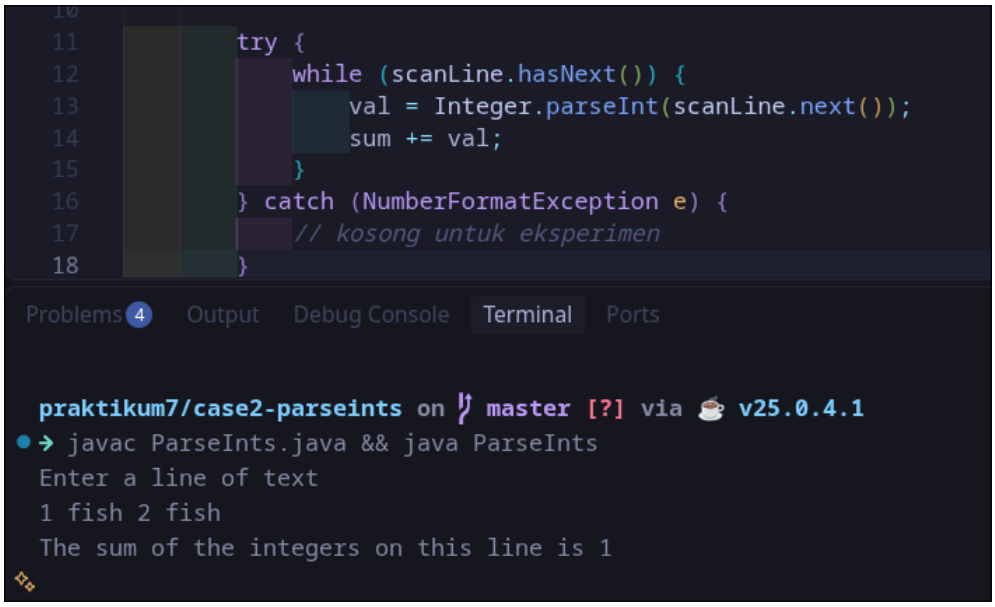
Sekarang seluruh perulangan while dibungkus satu try-catch, dengan catch yang sengaja dikosongkan.

```
try {
    while (scanLine.hasNext()) {
        val = Integer.parseInt(scanLine.next());
        sum += val;
    }
} catch (NumberFormatException e) {
    // kosong untuk eksperimen
}
```

Hasil percobaannya, proses berhenti di token bukan angka yang pertama. Input We have 2 dogs and 1 cat. menghasilkan sum = 0, sedangkan 1 fish 2 fish menghasilkan sum = 1.



Gambar 7. Try-catch di luar while, input We have 2 dogs and 1 cat.



Gambar 8. Try-catch di luar while, input 1 fish 2 fish

- Checkpoint 5

Blok try membungkus seluruh loop while. Begitu Integer.parseInt() melempar exception, alur langsung loncat keluar dari try menuju catch, dan loncatan itu mengakhiri while secara permanen. Itu sebabnya sisa token tidak pernah terbaca.

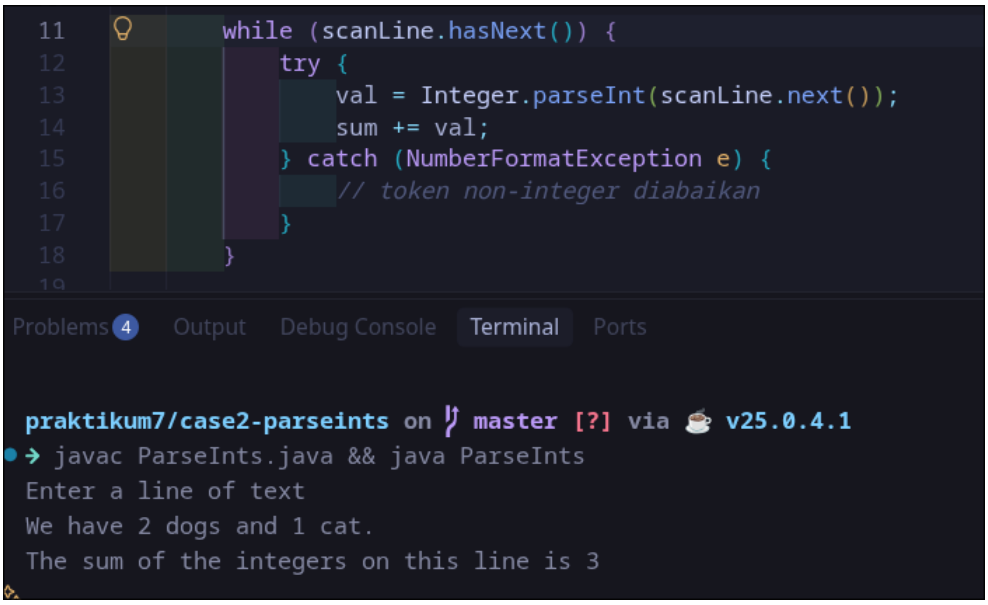
2.4 Try-Catch di Dalam While (T7-05)

Kali ini try-catch dipindah ke dalam badan while, jadi tiap token dicek sendiri-sendiri.

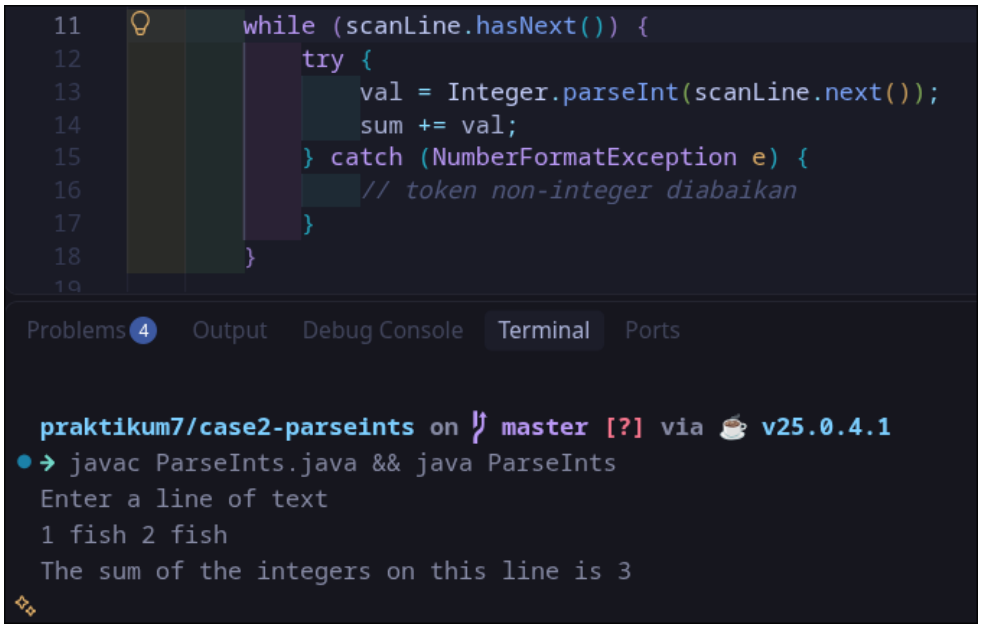
```
while (scanLine.hasNext()) {
    try {
        val = Integer.parseInt(scanLine.next());
        sum += val;
    } catch (NumberFormatException e) {
        // token non-integer diabaikan
    }
```

```
}
```

Sekarang semua token angka berhasil dijumlahkan. Input We have 2 dogs and 1 cat. menghasilkan sum = 3, begitu juga 1 fish 2 fish yang menghasilkan sum = 3.



Gambar 9. Try-catch di dalam while, input We have 2 dogs and 1 cat.



Gambar 10. Try-catch di dalam while, input 1 fish 2 fish

- Checkpoint 6

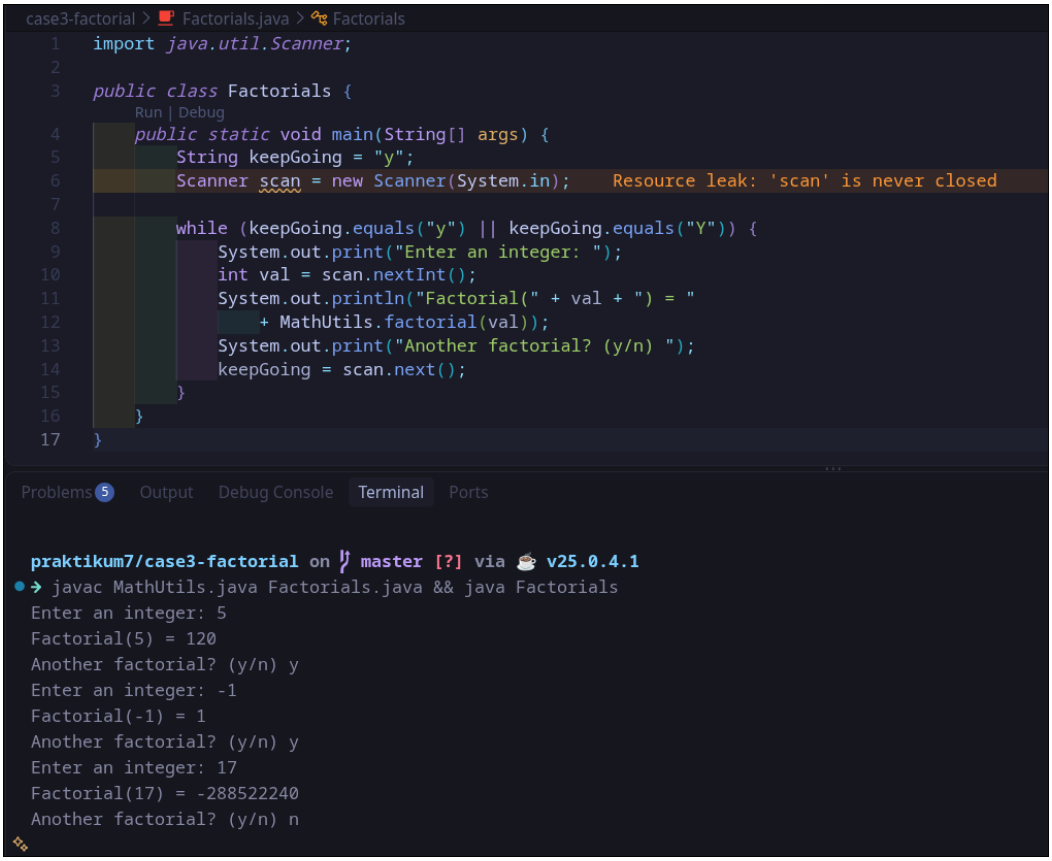
Perbandingan dua penempatan try-catch saya rangkum di tabel berikut.

Penempatan try-catch	Alur saat token tidak valid	Token berikutnya diproses?
Di luar while	Alur loncat keluar dari perulangan menuju catch.	Tidak
Di dalam while	Alur loncat ke catch hanya untuk iterasi itu saja.	Ya, perulangan lanjut ke iterasi berikutnya.

C. Factorials (Melempar Exception)

3.1 Kondisi Awal yang Belum Tervalidasi

Method MathUtils.factorial() yang awal belum memeriksa nilai input sama sekali. Input -1 menghasilkan 1, yang salah secara matematika. Input 17 menghasilkan -288522240, angka negatif karena terjadi integer overflow.



Gambar 11. Hasil factorial yang salah pada kondisi awal

- Checkpoint 7

Mengembalikan 1 untuk angka negatif berbahaya karena jadi kesalahan yang tidak kelihatan (silent error). Pemanggil mengira hasilnya benar padahal salah. Kalau method cuma mencetak pesan teks juga kurang aman, sebab method tetap harus mengembalikan nilai integer, dan nilai itu bisa saja terpakai lagi oleh fungsi lain tanpa disadari.

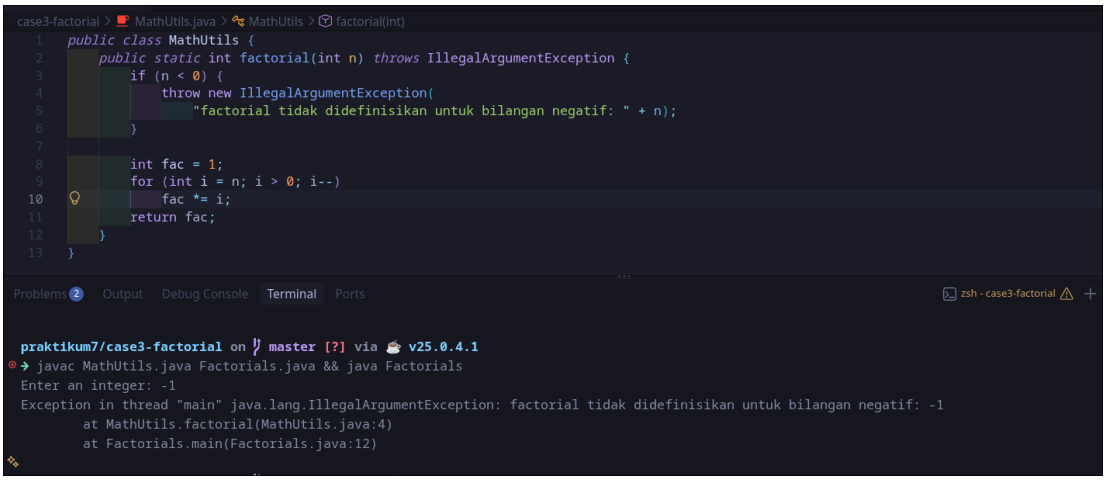
3.2 Melempar IllegalArgumentException (T7-06)

Method factorial saya ubah supaya melempar IllegalArgumentException kalau inputnya negatif.

```
public class MathUtils {
    public static int factorial(int n) throws
IllegalArgumentException {
        if (n < 0) {
            throw new IllegalArgumentException(
                "factorial tidak didefinisikan untuk bilangan
negatif: " + n);
        }
```

```
        int fac = 1;
        for (int i = n; i > 0; i--)
            fac *= i;
        return fac;
    }
}
```

Waktu diuji dengan -1, method menolak input itu dan melempar exception. Di tahap ini belum ada catch di pemanggilnya.



Gambar 12. Exception dilempar tanpa ada catch

- Checkpoint 8

Yang mendeteksi kondisi tidak valid adalah method MathUtils.factorial(). Yang belum menangani adalah pemanggilnya, yaitu Factorials.main(). Karena tidak ada yang menangkap, exception merambat naik sampai menghentikan thread utama program.

3.3 Penanganan Exception di Pemanggil (T7-07)

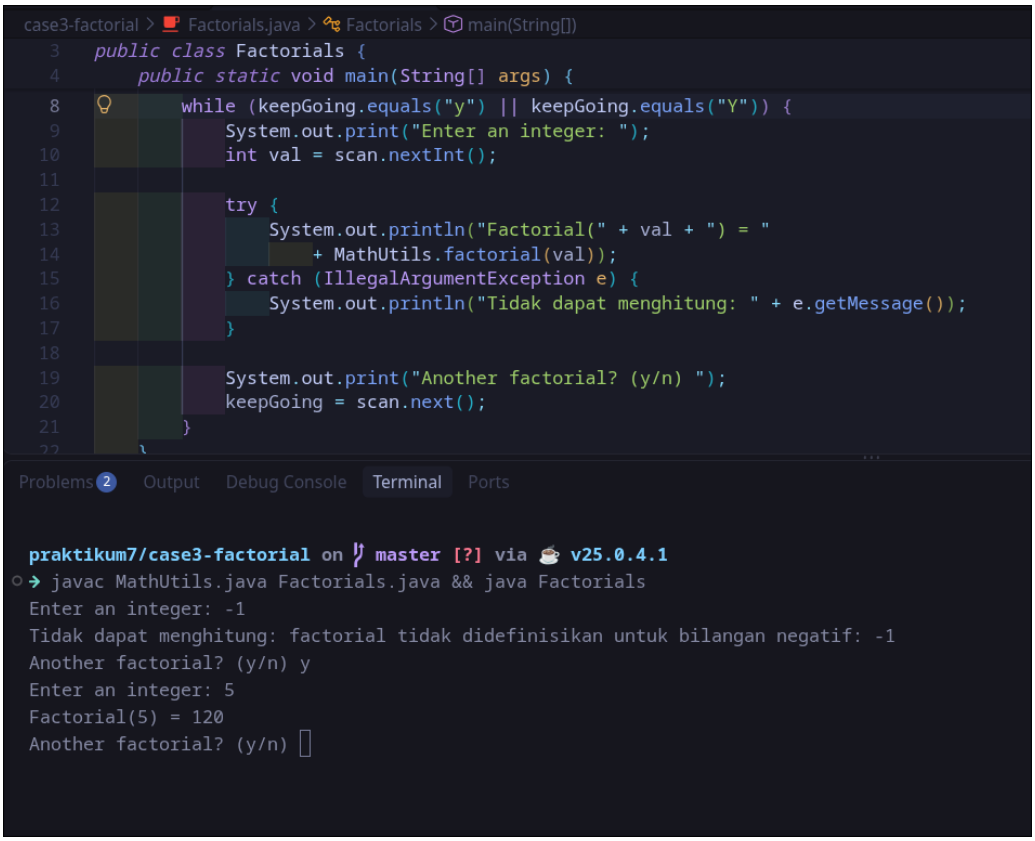
Berikutnya try-catch ditambahkan di Factorials.java, tepat di sekitar pemanggilan factorial.

```
while (keepGoing.equals("y") || keepGoing.equals("Y")) {
    System.out.print("Enter an integer: ");
    int val = scan.nextInt();

    try {
        System.out.println("Factorial(" + val + ") = "
            + MathUtils.factorial(val));
    } catch (IllegalArgumentException e) {
        System.out.println("Tidak dapat menghitung: " +
e.getMessage());
    }

    System.out.print("Another factorial? (y/n) ");
    keepGoing = scan.next();
}
```

Sekarang nilai negatif ditolak dengan pesan yang jelas, lalu program tetap bisa memproses input valid sesudahnya.



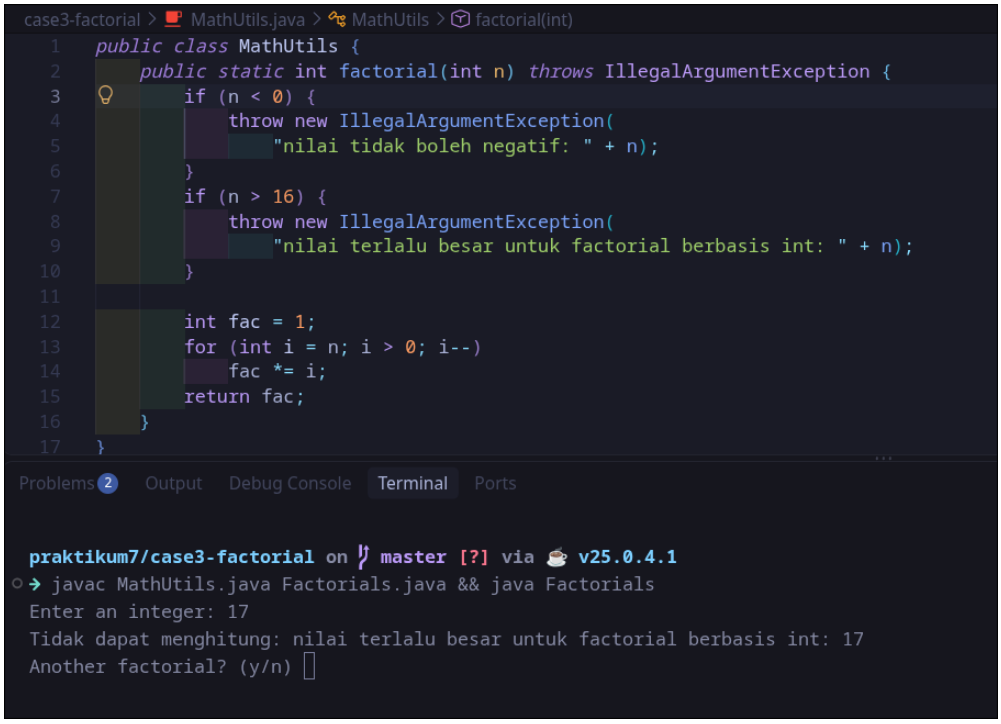
Gambar 13. Program pulih setelah exception ditangani

3.4 Batas Overflow Integer (T7-08)

Terakhir saya tambahkan pengecekan batas atas n > 16 di MathUtils.java.

```
if (n < 0) {
    throw new IllegalArgumentException(
        "nilai tidak boleh negatif: " + n);
}
if (n > 16) {
    throw new IllegalArgumentException(
        "nilai terlalu besar untuk factorial berbasis int: " + n);
}
```

Dengan input 17, program menampilkan pesan bahwa nilainya melewati batas dan perulangannya tetap jalan.



Gambar 14. Penanganan batas atas pada factorial

- Checkpoint 9

Pesan di exception menjelaskan penyebab kegagalan yang spesifik, jadi kelihatan bedanya apakah gagal karena nilai negatif atau karena nilainya terlalu besar untuk tipe data int.

D. Rekapitulasi Skenario Pengujian

4.1 Tabel Skenario T7-01 sampai T7-08

ID	Skenario	Hasil yang diharapkan	Bukti	Status
T7-01	CountLetters dengan satu kata	Frekuensi huruf tercetak normal	Gambar 1	PASS
T7-02	CountLetters sebelum catch	ArrayIndexOutOfBoundsException muncul	Gambar 2	PASS
T7-03	CountLetters setelah catch	Karakter non-huruf dilewati, proses lanjut	Gambar 4	PASS
T7-04	ParseInts, try-catch di luar while	Berhenti di token salah yang pertama	Gambar 7, Gambar 8	PASS
T7-05	ParseInts, try-catch di dalam while	Semua token diproses, angka dijumlahkan	Gambar 9, Gambar 10	PASS
T7-06	factorial(-1) sebelum ada catch	IllegalArgumentException menghentikan alur	Gambar 12	PASS
T7-07	factorial(-1) setelah catch di loop	Pesan tampil dan loop tetap lanjut	Gambar 13	PASS
T7-08	factorial(17)	Exception batas n > 16 tertangani	Gambar 14	PASS

E. Refleksi dan Kesimpulan

5.1 Jawaban Refleksi

- Kenapa CountLetters awal berhenti?

Karena karakter non-huruf menghasilkan indeks array negatif, contohnya -33 untuk spasi. Setelah ditambah blok catch, error itu ditangani dan perulangan jalan terus ke karakter berikutnya.

- Apakah exception selalu bikin aplikasi berhenti?

Tidak. Dari percobaan CountLetters dan ParseInts terlihat program bisa pulih dan menyelesaikan iterasinya sampai habis selama exception-nya ditangani.

- Beda letak try-catch pada ParseInts

Kalau di luar while, perulangan berhenti total saat exception terjadi. Kalau di dalam while, cuma token yang salah yang dilewati dan iterasi berikutnya tetap jalan.

- Kenapa melempar exception lebih aman daripada mengembalikan nilai semu?

Karena kesalahannya tidak tersembunyi. Pemanggil langsung tahu fungsinya gagal, jadi tidak ikut memakai hasil hitungan yang salah.

- Beda peran throw dan catch

throw dipakai untuk mendeteksi kondisi yang tidak valid lalu melempar sinyal kesalahan. catch menangkap sinyal itu dan menjalankan logika pemulihannya.

- Manfaat pesan yang berbeda pada IllegalArgumentException

Pesannya menjelaskan penyebab gagalnya dengan spesifik, apakah karena input negatif atau karena melewati batas kapasitas integer.

- Pertanyaan tambahan soal checked dan unchecked exception

Kenapa compiler Java mewajibkan penanganan atau deklarasi untuk checked exception, sedangkan untuk unchecked exception tidak diwajibkan?

5.2 Kesimpulan

Exception handling tidak sekadar mencegah program crash saja. Intinya mengendalikan alur pemulihan program supaya kegagalan bisa ditangani dengan aman, terprediksi, dan leboh informatif.